



Arrays, Matrices, and Linked Lists

CEN207 Data Structures — Week 2

Asst. Prof. Dr. Uğur CORUH · Fall 2026–2027

Today's plan (3 hours)

Hour	Topic
1	Arrays: insert/delete, dynamic growth Anim 1–2 · 2-D layout, rotation, rearrange Anim 3–5
2	Sparse matrices Anim 6–8 · lists: node/head/NULL, singly ops Anim 9–12
3	Doubly, circular, Josephus, XOR, skip list Anim 13–17 · arrays vs. lists

Learning outcomes: LO.1 (explain fundamental data structures) · LO.2 (analyze complexity) · LO.7 (choose the right structure)

This week's concepts — where

Concept	Where
Arrays: shifting, dynamic growth	Section 1
2-D arrays, rotation, rearrangement	Sections 2–3
Sparse matrices	Section 4
Singly/doubly/circular lists	Sections 5–8
XOR list, skip list, arrays vs. lists	Sections 9–11

How the code examples work

- Every idea has a complete **C** and **Java** program
- C: `gcc -std=c11 -Wall -Wextra -o /tmp/x file.c && /tmp/x`
- Java: `javac -d /tmp/j File.java && java -cp /tmp/j File`
- Sources: `code/week-02/c/` and `code/week-02/java/`
- Each program's expected output is in the week notes

Recap — Week 1: memory and pointers

- A variable is a labeled **box** in memory
- A pointer **stores an address** — it points at a box
- `malloc(n)` : reserve `n` bytes, returns an address
- `free(p)` : give the box back; then set `p = NULL`
- `NULL` : a pointer that points at **nothing**

Map of the week — at a glance

Arrays & matrices	Linked lists
Shifting, dynamic growth	Node, head, NULL
Row-major, rotation, rearrange	Singly, doubly, circular
Sparse: triplet, transpose, add	Josephus, XOR, skip list

1. Arrays in Memory: Insertion, Deletion, Growth

A question to start

An array is `n` boxes in a row. Insert one value in the middle — what has to move before the new value can even go in?

Intuition — a row of numbered lockers

- Each box sits at $\text{index} \times \text{box size}$ from the start
- `arr[k]` is pure arithmetic — no searching needed
- But lockers do not slide apart by themselves
- Making room means moving every locker after it

size vs. capacity

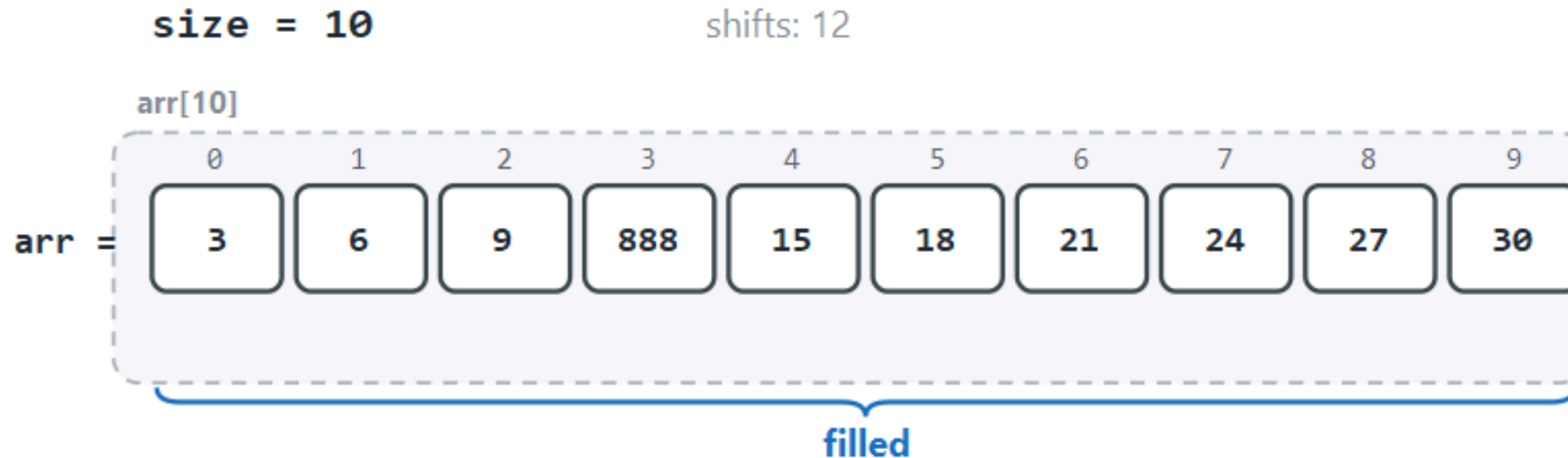
- A fixed array reserves `CAP` boxes up front
- `size` tracks how many boxes are **actually used**
- `insert_at(k, v)` : make room at index `k`
- `delete_at(k)` : close the gap at index `k`

Array insert and delete, step by step



Done: size = 9, 24 shifts in total. $k = 0$ is the slowest insert ($O(n)$), $k = \text{size}$ is the fastest ($O(1)$).

Edge case — overflow: the array is full



Done: size = 10, 12 shifts in total, 2 overflows. $k = 0$ is the slowest insert ($O(n)$), $k = \text{size}$ is the fastest ($O(1)$).

Code — `insert_at()`

```
bool insert_at(int k, int v) {  
    if (size == CAP)  
        return false;          /* full: overflow */  
    for (int i = size; i > k; i--)  
        arr[i] = arr[i - 1];    /* shift right */  
    arr[k] = v;  
    size++;  
    return true;  
}
```

Code — delete_at()

```
bool delete_at(int k) {  
    if (size == 0)  
        return false;    /* empty: underflow */  
    for (int i = k; i < size - 1; i++)  
        arr[i] = arr[i + 1]; /* shift left */  
    size--;  
    return true;  
}
```

Complexity and common mistakes

- `k = 0` : shifts **every** element — $O(n)$
- `k = size` (append): no shift at all — $O(1)$
- Mistake: forgetting the overflow/underflow check first
- Mistake: shifting in the wrong direction, overwriting data

Mini-quiz

You insert 5 values, always at index 0.
How many total element-shifts happen?

Answer

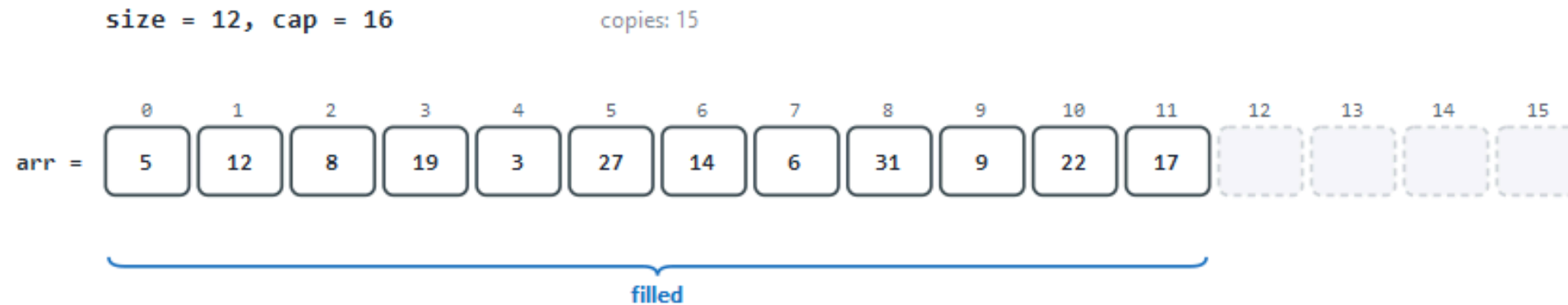
$0+1+2+3+4 = 10$ shifts. Each front-insert shifts every element already there — the classic $O(n)$ worst case, repeated five times.

But what if the size is unknown?

A fixed array's **CAP** is a hard ceiling.

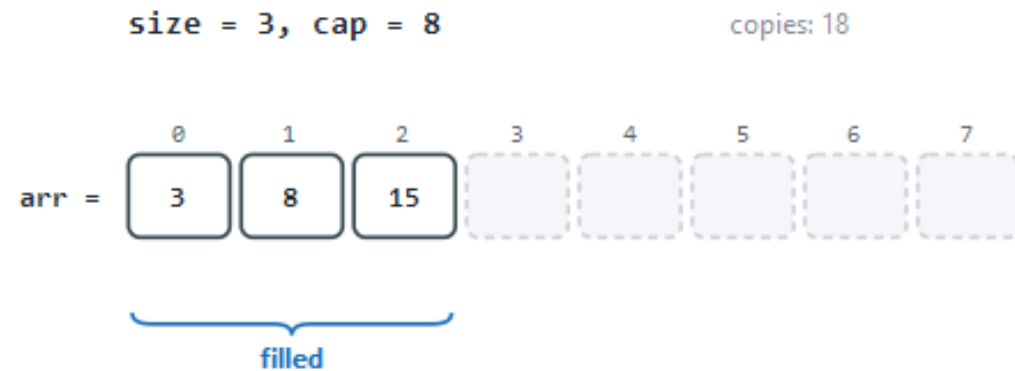
A **dynamic array** grows its own capacity on demand, instead of rejecting the insert.

Dynamic array growth, step by step



Done: 12 appends, 4 growths, 15 copies in total (fewer than $2 \cdot 12 = 24$). A single append can cost $O(n)$, but the AMORTIZED cost is $O(1)$.

Edge case — shrinking back down



Done: 12 appends, 3 growths, 1 shrink, 18 copies in total (fewer than $2 \cdot 12 = 24$). A single append can cost $O(n)$, but the AMORTIZED cost is $O(1)$.

Code — `da_resize()`

```
static void da_resize(DynArray *a, int new_cap) {
    int *fresh = malloc(new_cap * sizeof(int));
    for (int i = 0; i < a->size; i++)
        fresh[i] = a->data[i];    /* copy every value */
    free(a->data);                /* old block freed */
    a->data = fresh;
    a->cap = new_cap;
}
```

Code — da_append()

```
void da_append(DynArray *a, int v) {  
    if (a->size == a->cap) {  
        int new_cap = a->cap * 2;    /* growth factor 2 */  
        da_resize(a, new_cap);  
    }  
    a->data[a->size++] = v;  
}
```

Complexity and common mistakes

- A growing append copies `size` elements: $O(n)$
- Most appends just write one slot: $O(1)$
- Over `n` appends, total copies stay **under $2n$**
- So the **amortized** cost per append is $O(1)$
- Mistake: treating every single append as $O(n)$

Mini-quiz

Starting from `cap = 1` and doubling, how many times does the array grow while 100 values are appended one by one?

Answer

7 times ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$).

$\log_2(100) \approx 6.6$, rounded up — growth is **logarithmic** in the final size, not linear.

2. 2-D Arrays: Row-Major vs Column-Major

A question to start

Memory is one long row of bytes. A matrix looks two-dimensional — rows and columns. How does `mat[i][j]` become one address?

Intuition — a bookshelf, one shelf at a time

- Picture the matrix's rows laid end to end
- Row 0's cells, then row 1's cells, and so on
- That layout is called **row-major** order
- Some languages (Fortran, MATLAB) lay out **columns** first

The address formulas

- **Row-major:** $\text{addr}(i, j) = i * \text{COLS} + j$
- **Column-major:** $\text{addr}(i, j) = j * \text{ROWS} + i$
- C and Java: row-major; Fortran, MATLAB: column-major
- The formula is arithmetic — no searching at all

Matrix in memory, step by step

	0	1	2	3
M[0] =	8	16	24	32
M[1] =	40	48	56	64
M[2] =	72	80	88	96

	0	1	2	3	4	5	6	7	8	9	10	11
mem =	8	16	24	32	40	48	56	64	72	80	88	96
	row 0				row 1				row 2			

	0	1	2	3	4	5	6	7	8	9	10	11
order =	8	16	24	32	40	48	56	64	72	80	88	96

Done: 12 cells, 11/11 steps adjacent ($\Delta=1$), the biggest jump is $\Delta=1$. The traversal MATCHES the storage order, so it is cache-friendly.

Edge case — column-major storage, row-by-row walk

	0	1	2	3	4
M[0] =	4	9	-3	16	21
M[1] =	7	-12	25	2	18
M[2] =	-6	14	8	-20	30

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
mem =	4	7	-6	9	-12	14	-3	25	8	16	2	-20	21	18	30
	col 0			col 1			col 2			col 3			col 4		

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
order =	4	9	-3	16	21	7	-12	25	2	18	-6	14	8	-20	30

Done: 15 cells, 0/14 steps adjacent ($\Delta=1$), the biggest jump is $\Delta=11$. The traversal goes AGAINST the storage order, which is BAD for the cache.

Code — `addr()` and a row-major walk

```
int mat[ROWS][COLS];  
/* address of mat[i][j], in ints from the start */  
int addr(int i, int j) {  
    return i * COLS + j;    /* row-major */  
}  
  
void traverse_row_major(int mat[ROWS][COLS]) {  
    for (int i = 0; i < ROWS; i++)  
        for (int j = 0; j < COLS; j++)  
            visit(mat[i][j]);  
}
```

Complexity and common mistakes

- Address lookup: pure arithmetic — $O(1)$
- Row-major walk, row-major storage: every step $\Delta=1$
- Wrong layout for the walk: a jump **every** step
- Mistake: assuming every language is row-major

Mini-quiz

A row-major matrix has 5 columns. `mat[3][2]` sits at address 17. What is `mat[3][3]` 's address?

Answer

18. Moving one column right adds exactly **1** in row-major order — the next cell is always the very next address.

3. Rotation and Rearrangement: Two Pointers

A question to start

Rotate a 12-value array left by 4, using **no** second array. Where would you even start, with only $O(1)$ extra space allowed?

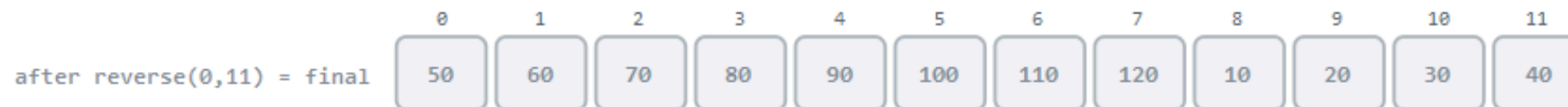
Intuition — reversal, three times

- Reverse the first part, reverse the rest
- Then reverse the **whole** array once more
- Three reversals land every value exactly right
- No second array, no shifting one step at a time

rotate_left : three reversals

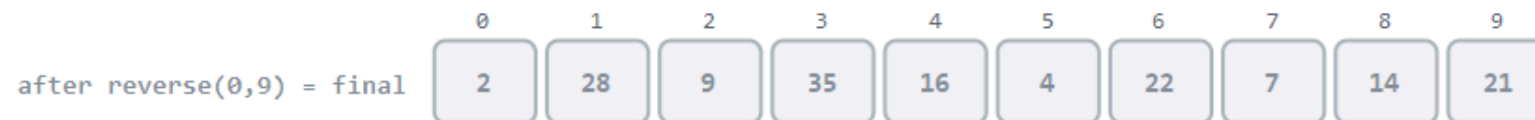
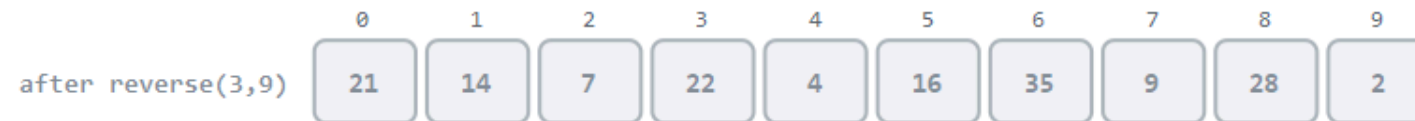
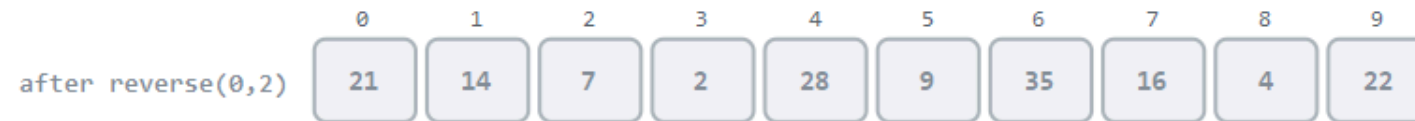
- $d = d \% n$ — the real rotation amount
- Reverse `arr[0..d-1]`
- Reverse `arr[d..n-1]`
- Reverse `arr[0..n-1]` — done

Array rotation, step by step



Done: rotating left by $d=4$ took 12 swaps, $O(n)$ time. Result: 50, 60, 70, 80, 90, 100, 110, 120, 10, 20, 30, 40.

Edge case — d larger than n



Done: rotating left by d=23 took 9 swaps, O(n) time. Result: 2, 28, 9, 35, 16, 4, 22, 7, 14, 21.

Code — reverse()

```
void reverse(int arr[], int lo, int hi) {  
    while (lo < hi) {  
        int tmp = arr[lo];  
        arr[lo] = arr[hi];  
        arr[hi] = tmp;  
        lo++;  
        hi--;  
    }  
}
```

Code — rotate_left()

```
void rotate_left(int arr[], int n, int d) {  
    if (n == 0) return;          /* empty: nothing to do */  
    d = d % n;  
    reverse(arr, 0, d - 1);      /* first d */  
    reverse(arr, d, n - 1);      /* the rest */  
    reverse(arr, 0, n - 1);      /* whole array */  
}
```

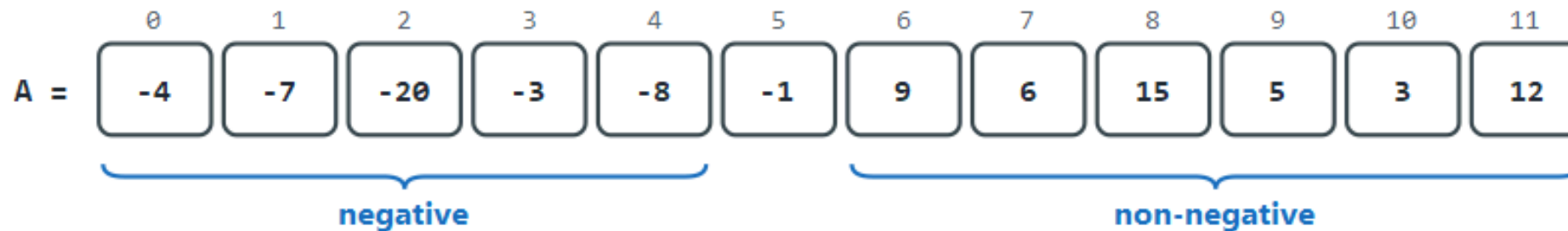
Complexity and common mistakes

- Three passes over the array, each $O(n)$ — still $O(n)$
- $O(1)$ extra space: no second array anywhere
- Mistake: forgetting `d = d % n` first
- Mistake: off-by-one bounds inside `reverse`

Rearranging: two pointers close in

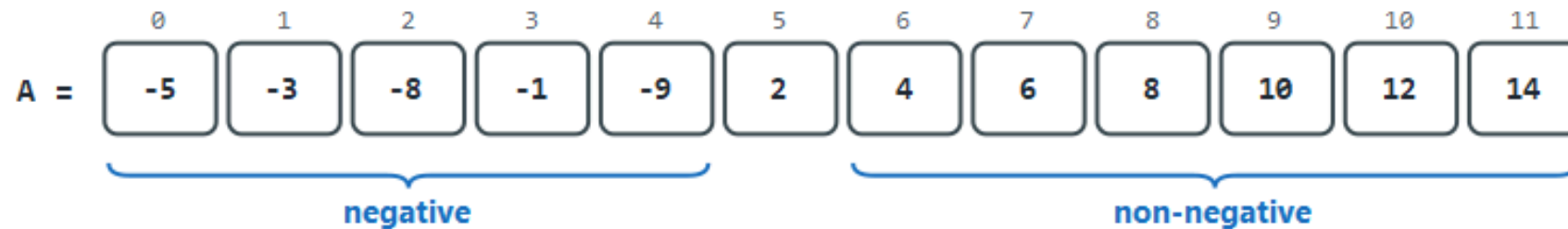
- Goal: negatives left, non-negatives right
- `left` skips values already negative
- `right` skips values already non-negative
- When both stop, swap and step inward

Array rearrangement, step by step



Done: 3 swaps gathered the negatives on the left and the non-negatives on the right: -4, -7, -20, -3, -8, -1, 9, 6, 15, 5, 3, 12. A single $O(n)$ pass.

Edge case — already segregated



Done: 0 swaps gathered the negatives on the left and the non-negatives on the right: -5, -3, -8, -1, -9, 2, 4, 6, 8, 10, 12, 14. A single $O(n)$ pass.

Code — segregate()

```
void segregate(int arr[], int n) {  
    int left = 0, right = n - 1;  
    while (left < right) {  
        while (left < right && arr[left] < 0) left++;  
        while (left < right && arr[right] >= 0) right--;  
        if (left < right) {  
            int tmp = arr[left];  
            arr[left] = arr[right];  
            arr[right] = tmp;  
            left++; right--;  
        }  
    }  
}
```

Complexity and common mistakes

- Both pointers move **inward only** — $O(n)$ total
- One pass, $O(1)$ extra space
- Mistake: treating `0` as negative — it is **not**
- Mistake: forgetting the `left < right` guard inside

Mini-quiz

After `segregate`, is the array **sorted** within each half, or just split in two?

Answer

Just split. Negatives end up left of non-negatives, but neither half is sorted — **segregate** never compares values to order them.

4. Sparse Matrices

A question to start

A 1000×1000 matrix has a million cells.
Only 200 are nonzero. Why store the other
999,800 zeros at all?

Intuition — a mostly-empty parking lot

- Most spaces sit empty, all day, every day
- A clipboard listing only the **occupied** spots suffices
- Position + value is all you need to remember
- That clipboard is the **triplet** representation

The triplet: (row, col, value)

- One entry per **nonzero** cell, nothing else
- Scan row-major, skip every zero found
- A sparse matrix's triplet table can be tiny
- Rebuilding the dense matrix just needs the triplets

Sparse matrix as triplets, step by step

	0	1	2	3	4	5
M[0] =	0	0	3	0	0	0
M[1] =	0	0	0	0	4	0
M[2] =	0	5	0	0	0	0
M[3] =	0	0	0	0	0	7
M[4] =	6	0	0	8	0	0

rows=5, cols=6, count=6

row	col	value
0	2	3
1	4	4
2	1	5
3	5	7
4	0	6
4	3	8

Done: only 6 of 30 cells are nonzero (%80 would be wasted on zeros). The triplet table has 6 rows -- far smaller than the dense matrix.

Edge case — an all-zero matrix

	0	1	2	3
M[0] =	0	0	0	0
M[1] =	0	0	0	0
M[2] =	0	0	0	0
M[3] =	0	0	0	0

rows=4, cols=4, count=0

row col value

Done: only 0 of 16 cells are nonzero (%100 would be wasted on zeros). The triplet table has 0 rows -- far smaller than the dense matrix.

Code — `to_triplets()`

```
int to_triplets(int mat[ROWS][COLS], Triplet out[]) {
    int k = 0;
    for (int i = 0; i < ROWS; i++)
        for (int j = 0; j < COLS; j++)
            if (mat[i][j] != 0) {
                out[k].row = i;
                out[k].col = j;
                out[k].value = mat[i][j];
                k++;
            }
    return k;
}
```

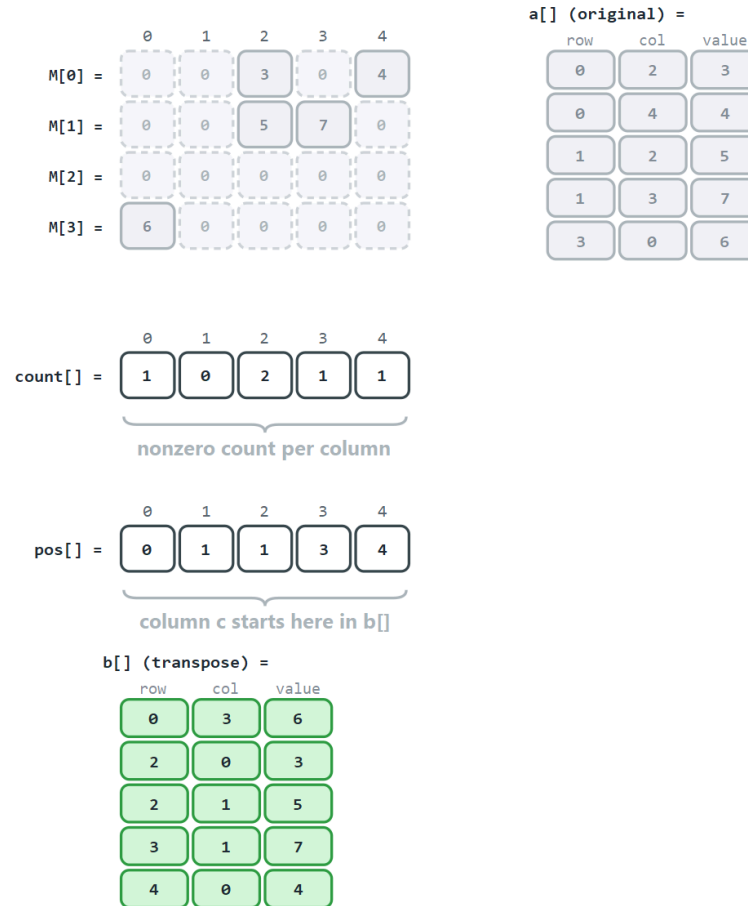
Complexity

- Scanning the dense matrix: $O(\text{rows} \times \text{cols})$
- Output size equals the **nonzero count**, never more
- A very sparse matrix: triplets are far smaller

Fast transpose: no re-sorting needed

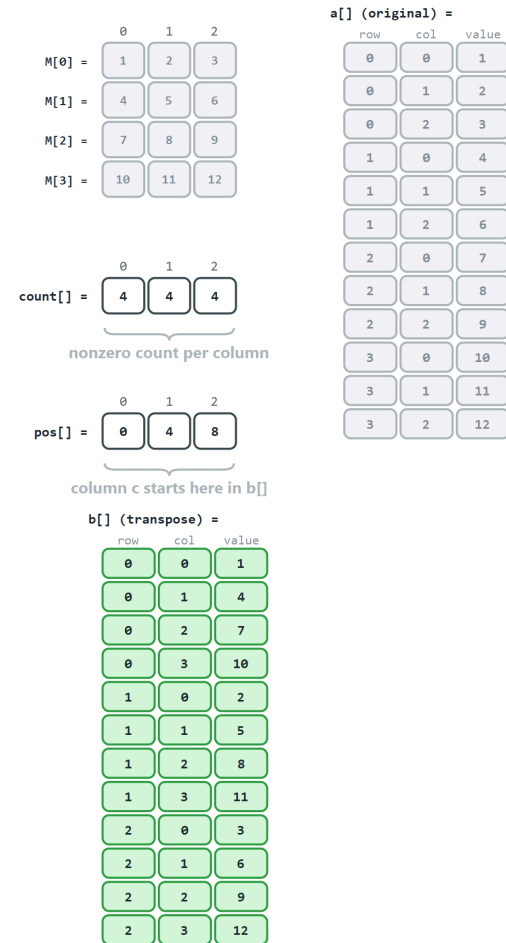
- Naive transpose: swap `(row, col)`, then **sort** — $O(\text{nnz} \log \text{nnz})$
- Better: **count** nonzeros per column first
- Turn counts into starting **positions** (a prefix sum)
- One more pass places every triplet, already sorted

Fast transpose, step by step



Done: 5 triplets landed in b[] with just one extra pass ($O(nnz + COLS)$) -- much faster than a general sort ($O(nnz \log nnz)$).

Edge case — a fully dense matrix



Done: 12 triplets landed in b[] with just one extra pass ($O(nnz + COLS)$) -- much faster than a general sort ($O(nnz \log nnz)$).

Code — counting and prefix sums

```
int count[COLS] = {0};
int pos[COLS];
for (int i = 0; i < nnz; i++)
    count[a[i].col]++;          /* per column */
pos[0] = 0;
for (int c = 1; c < COLS; c++)
    pos[c] = pos[c - 1] + count[c - 1];
```

Code — placing every triplet

```
for (int i = 0; i < nnz; i++) {  
    int c = a[i].col;  
    int p = pos[c]++;  
    b[p].row = a[i].col;    /* row/col swap */  
    b[p].col = a[i].row;  
    b[p].value = a[i].value;  
}
```

Complexity and common mistakes

- Two passes, $O(\text{nnz} + \text{COLS})$ — no sorting at all
- Far faster than a general sort's $O(\text{nnz} \log \text{nnz})$
- Mistake: skipping the prefix sum — output lands unsorted
- Mistake: forgetting to swap `row` and `col`

Adding two sparse matrices: a merge

- Both triplet lists already row-major **sorted**
- Walk both together, like merge sort's merge step
- Earlier `(row,col)` wins and is copied through
- Same `(row,col)` in both: **add** the values

Sparse matrix addition, step by step

a[] =

row	col	val
0	0	4
0	3	2
1	1	5
2	0	3
2	2	7
2	3	1

b[] =

row	col	val
0	0	6
0	1	3
1	1	-2
1	3	8
2	1	4
2	3	9

sum =

row	col	val
0	0	10
0	1	3
0	3	2
1	1	3
1	3	8
2	0	3
2	1	4
2	2	7
2	3	10

Done: 6 + 6 triplets merged in one pass into 9 result triplets (3 cancelled/merged away). $O(n_a + n_b)$, no re-sorting.

Edge case — values that cancel to zero

a[] =

row	col	val
0	0	5
0	2	3
1	1	4
1	2	-6
2	0	2
2	1	9

b[] =

row	col	val
0	0	-5
0	1	7
1	1	6
1	2	6
2	0	-2
2	2	9

sum =

row	col	val
0	1	7
0	2	3
1	1	10
2	1	9
2	2	9

Done: 6 + 6 triplets merged in one pass into 5 result triplets (7 cancelled/merged away). $O(na + nb)$, no re-sorting.

Code — the merge loop

```
while (i < na && j < nb) {  
    if (/* ... a[i]'s (row,col) comes first ... */) {  
        out[k++] = a[i++];  
    } else if (/* ... b[j]'s comes first ... */) {  
        out[k++] = b[j++];  
    } else {  
        int sum = a[i].value + b[j].value;  
        if (sum != 0) /* ... out[k++] = sum entry ... */;  
        i++; j++;  
    }  
}
```

Complexity and common mistakes

- One merge pass: $O(na + nb)$, no re-sorting
- Never touches a cell absent from **both** lists
- Mistake: forgetting to drop a cell when the sum is 0
- Mistake: assuming unsorted triplet lists still merge correctly

Mini-quiz

Cell (1,2) holds 6 in matrix A and -6 in matrix B. What ends up in the sum's triplet list?

Answer

Nothing. $6 + (-6) = 0$, and a zero-sum cell is dropped entirely — the triplet list only ever holds nonzero values.

5. Linked Lists: Node, Head, NULL

A question to start

Every insert into an array can shift up to n elements. Is there a structure where inserting **never** moves existing values?

A short history

- **1956** — John McCarthy, Lisp's **cons cell**
- A cons cell: a value, plus a pointer to the next
- Six decades later, every language still uses it
- C's `struct Node { data; next; }` is the same idea

Intuition — a treasure hunt

- Each clue tells you **where** the next clue is
- You never see the whole map at once
- Follow one pointer, arrive, read, follow the next
- The **last** clue says "nothing here" — `NULL`

The whole idea, one struct

```
typedef struct Node {  
    int data;  
    struct Node *next;  
} Node;
```

- One value, one pointer — that is a linked list
- No shifting: a new node just gets a pointer

head and the NULL terminator

- head is the **only** fixed reference into the list
- Every other node is reached by following next
- The last node's next is NULL — "no more nodes"
- An empty list is just head == NULL

The "inception": a struct containing itself

- `struct Node` has a field of type `struct Node *`
- That is **not** infinite recursion — it is a pointer
- A pointer's size is fixed, known before `Node` is complete
- This is why `struct Node *next;` compiles, but `Node next;` cannot

Common mistakes

- Forgetting to check for `NULL` before `->next`
- Confusing a **node** (the struct) with a **pointer** to it
- Writing `Node next;` where a pointer was needed

Mini-quiz

Why must `next` be declared as `struct Node *`, never as a plain `struct Node`?

Answer

A plain `Node` field would need the **size** of `Node` to already be known — but `Node` is not finished being defined yet. A pointer's size never depends on that.

6. Singly Linked Lists: Insert, Delete, Search, Reverse

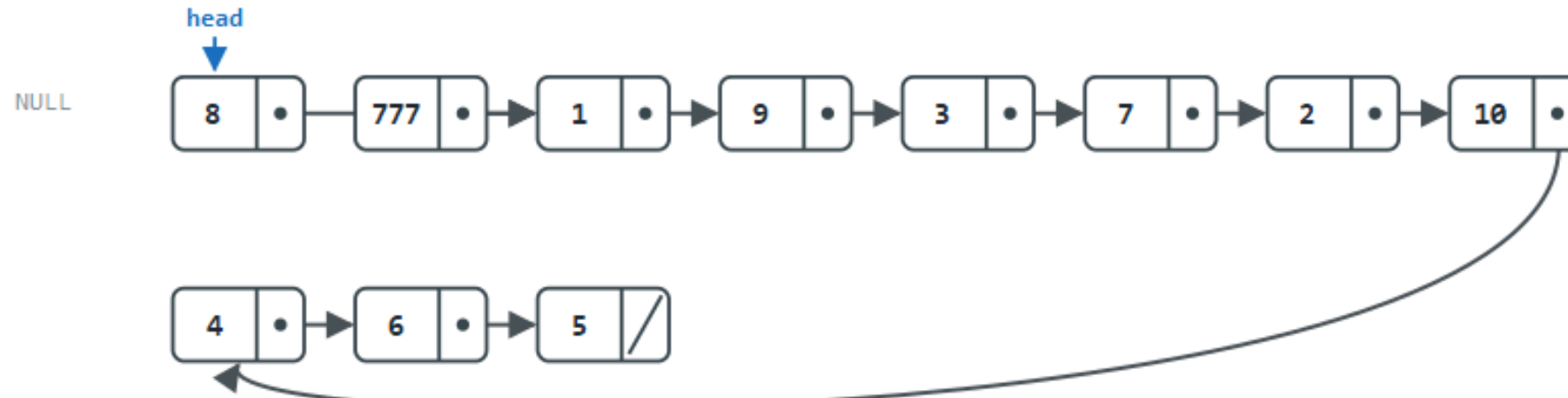
A question to start

Three insert positions — head, tail, and "after a given node" — cost very different amounts. Which is cheap, and which is not?

Three ways to insert

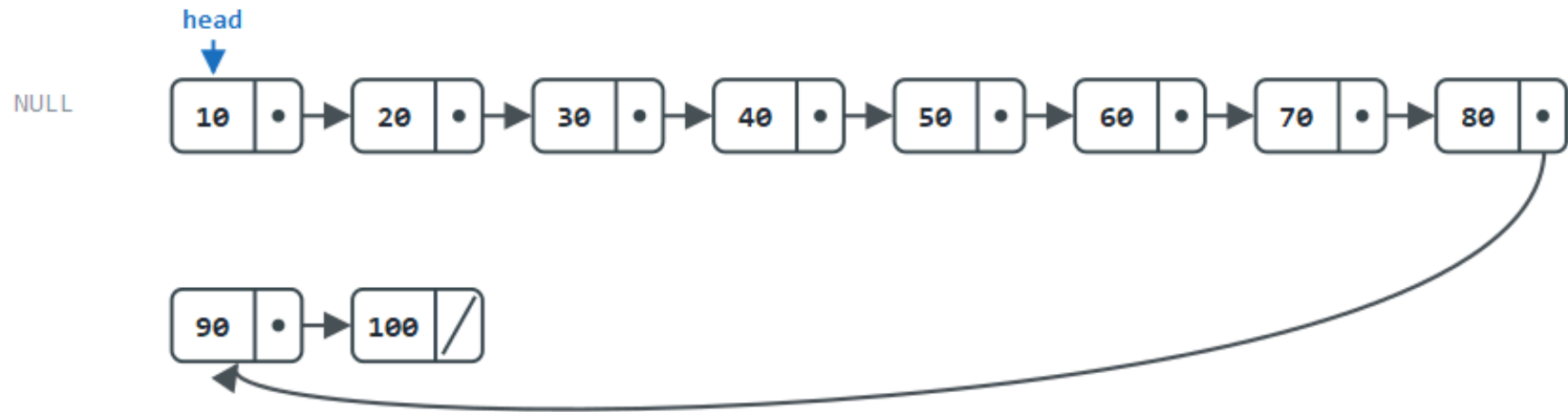
- `insert_head` : new node points at the old head — **$O(1)$**
- `insert_tail` : no tail pointer here — walk to the end
- `insert_after(prev, v)` : two pointer writes, **in order**
- Getting that order backward **breaks** the list

Singly-list insertion, step by step



Done: 5 insert at head, 5 at tail, 1 after a given node. List (head to tail): 8, 777, 1, 9, 3, 7, 2, 10, 4, 6, 5.

Edge case — the two steps, swapped



Done: 0 insert at head, 10 at tail, 0 after a given node. List (head to tail): 10, 20, 30, 40, 50, 60, 70, 80, 90, 100.

Code — `insert_head()`

```
Node *insert_head(Node *head, int value) {  
    Node *n = malloc(sizeof(Node));  
    n->data = value;  
    n->next = head;    /* points at the old head */  
    return n;          /* new node is head now */  
}
```

Code — `insert_after()` : order matters

```
void insert_after(Node *prev, int value) {  
    Node *n = malloc(sizeof(Node));  
    n->data = value;  
    n->next = prev->next;    /* STEP 1: new first */  
    prev->next = n;          /* STEP 2: then link */  
}
```

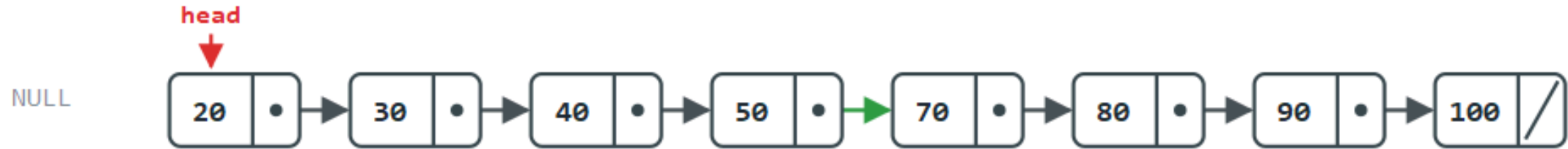
Complexity and common mistakes

- `insert_head`: $O(1)$ — no walking at all
- `insert_tail` (no tail pointer): walks to the end — $O(n)$
- `insert_after`: $O(1)$ once `prev` is already found
- Mistake: swapping STEP 1 and STEP 2 — cuts the list

Deleting by value

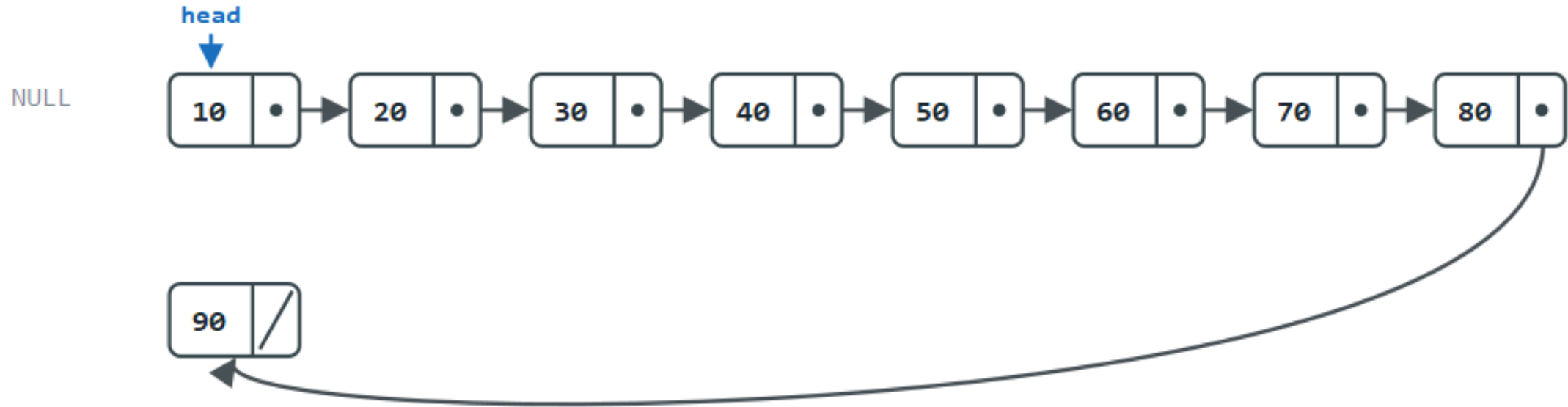
- Special case: the value is at the **head**
- Otherwise: `prev` / `cur` walk together, searching
- Found: `prev->next = cur->next` — the **bypass arrow**
- Then `free(cur)` — the node is gone

Singly-list deletion, step by step



Done: 2 successful deletions, 0 not found. List (head to tail): 20, 30, 40, 50, 70, 80, 90, 100.

Edge case — deleting the tail, then a missing value



Done: 1 successful deletion, 1 not found. List (head to tail): 10, 20, 30, 40, 50, 60, 70, 80, 90.

Code — delete_value()

```
Node *delete_value(Node *head, int value,
                    bool *removed) {
    *removed = false;
    if (head == NULL) return NULL;
    if (head->data == value) { /* delete head */
        Node *tmp = head;
        head = head->next;
        free(tmp);
        *removed = true;
        return head;
    }
    /* ... prev/cur scan, bypass, free(cur) ... */
}
```

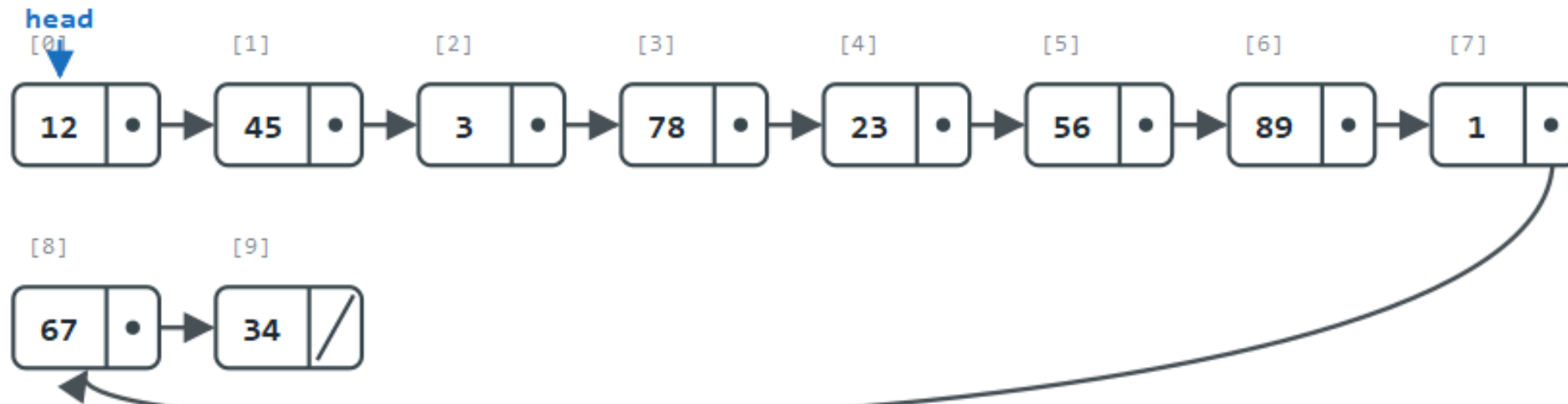
Complexity and common mistakes

- Deleting the head: $O(1)$, no search needed
- Deleting anywhere else: $O(n)$ to find it
- Mistake: forgetting the head is a **special** case
- Mistake: advancing `cur` before saving `prev`

Linear search: no shortcuts

- No index arithmetic — a list has no `arr[k]`
- Walk from `head`, comparing one node at a time
- Found: return its **position**; exhausted: return `-1`

Singly-list search, step by step



Done: 1 search. Results (index, comparisons): (4, 5).

Edge case — searching an empty list



Done: 1 search. Results (index, comparisons): (-1, 0).

Code — search()

```
int search(Node *head, int value) {  
    int index = 0;  
    for (Node *cur = head; cur != NULL;  
         cur = cur->next) {  
        if (cur->data == value)  
            return index;    /* found here */  
        index++;  
    }  
    return -1;                /* not found */  
}
```

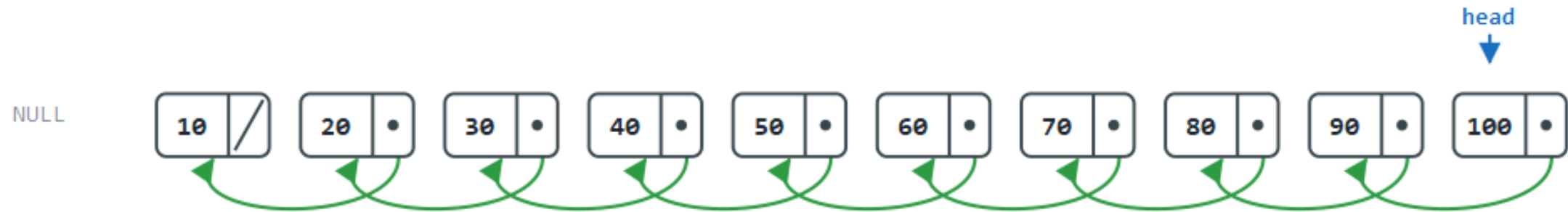
Complexity

- Every search: $O(n)$ — no index to jump to
- Contrast: `arr[k]` on an array is $O(1)$
- This exact gap drives the section 11 comparison

Reversing in place: three pointers

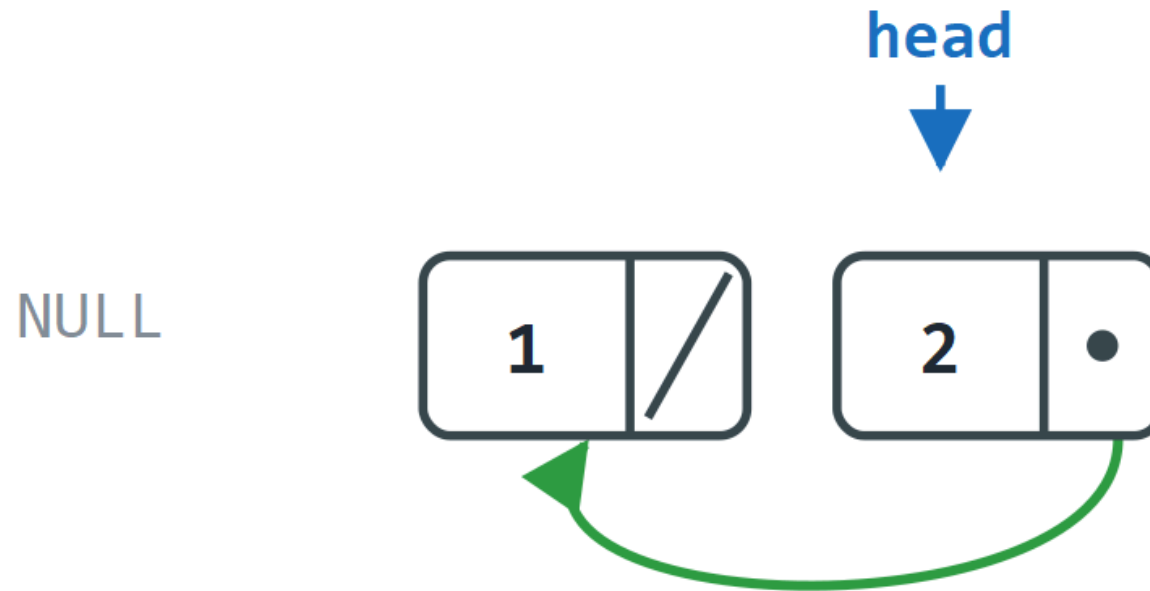
- `prev` trails, `curr` leads, `next` looks ahead
- Save `next` **before** overwriting `curr->next`
- Flip `curr->next` to point at `prev`
- Both pointers then step one node forward

Singly-list reversal, step by step



`curr == NULL`: the loop is done. `prev` points at the old last node, which is the new head; `head` now points there too.

Edge case — just two nodes



`curr == NULL`: the loop is done. `prev` points at the old last node, which is the new head; `head` now points there too.

Code — reverse()

```
Node *reverse(Node *head) {  
    Node *prev = NULL;  
    Node *curr = head;  
    while (curr != NULL) {  
        Node *next = curr->next; /* save rest */  
        curr->next = prev;      /* flip arrow */  
        prev = curr;  
        curr = next;  
    }  
    return prev;                /* new head */  
}
```

Complexity

- One pass, one flip per node — $O(n)$ time
- No extra nodes allocated — $O(1)$ space
- Compare: reversing an array also needs no extra pointers

Mini-quiz

If you flip `curr->next` to `prev` **before** saving `curr->next` into a temporary `next` variable, what breaks?

Answer

The rest of the list is lost. Once `curr->next` points backward, there is no way left to reach the nodes that used to follow it.

7. Doubly Linked Lists

A question to start

A singly list can only walk **forward**.

What has to change to walk backward too?

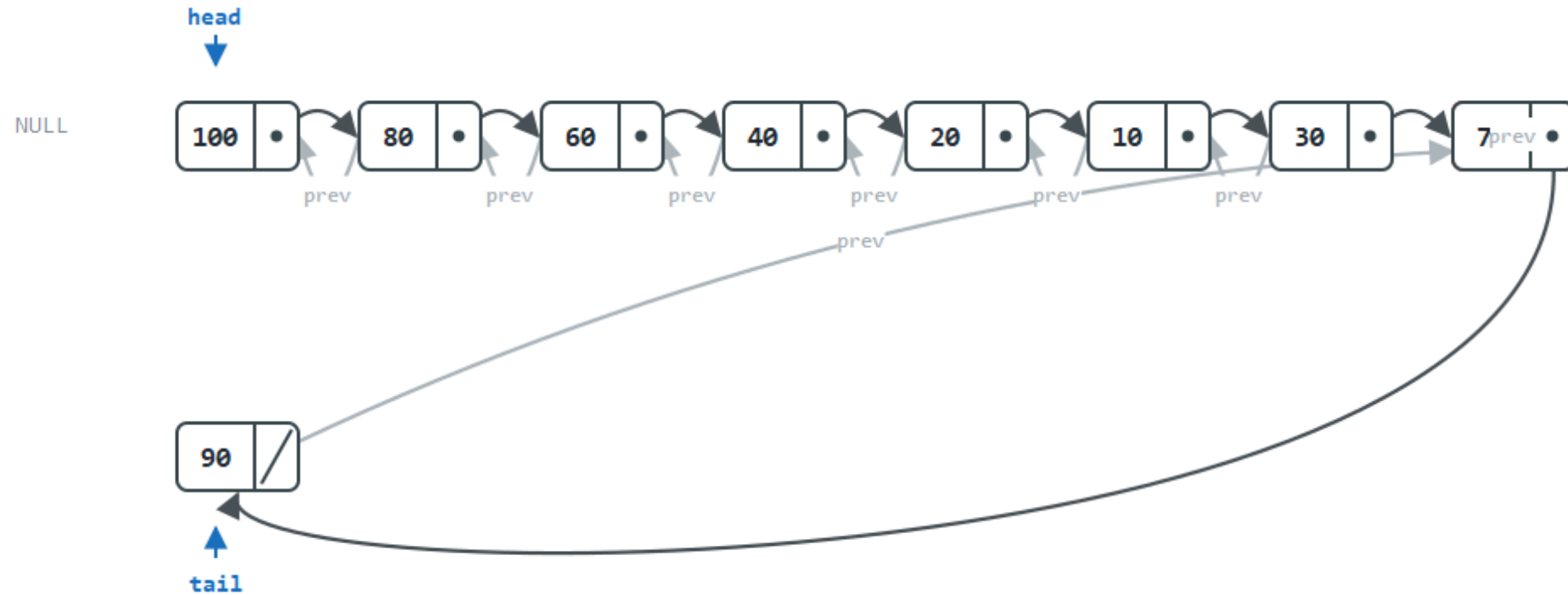
Intuition — a two-way street

- Singly list: a one-way street, forward only
- Doubly list: a two-way street, both directions
- Every node carries **two** signs: `next` and `prev`
- Turning around costs nothing extra — just follow `prev`

insert_after : four pointers to fix, not two

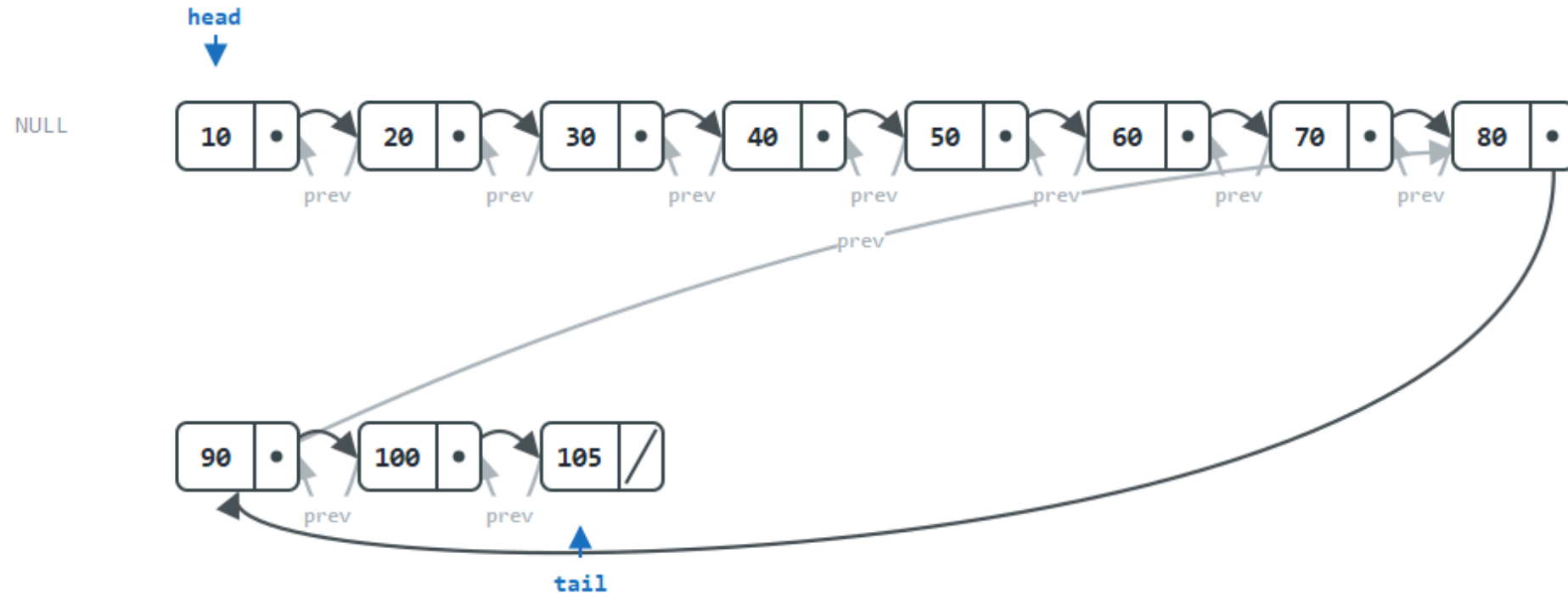
- Singly list's insert_after : fix 2 pointers
- Doubly list: also fix the new node's prev
- And the old next's prev, if it exists
- If cur was the tail, update list->tail too

Doubly-list insertion, step by step



Done: 10 insertions, 1 deletion (0 not found). Forward: 100, 80, 60, 40, 20, 10, 30, 70, 90. Backward: 90, 70, 30, 10, 20, 40, 60, 80, 100.

Edge case — inserting right after the tail



Done: 11 insertions, 0 deletions (0 not found). Forward: 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 105. Backward: 105, 100, 90, 80, 70, 60, 50, 40, 30, 20, 10.

Code — insert_after()

```
bool insert_after(List *list, int target, int v) {
    for (Node *cur = list->head; cur; cur = cur->next) {
        if (cur->data == target) {
            Node *n = malloc(sizeof(Node));
            n->data = v; n->prev = cur; n->next = cur->next;
            if (cur->next) cur->next->prev = n;
            else list->tail = n; /* cur was the tail */
            cur->next = n;
            return true;
        }
    }
    return false;
}
```

Code — `delete_value()` : no scan for `prev`

```
bool delete_value(List *list, int value) {
    for (Node *cur = list->head; cur; cur = cur->next) {
        if (cur->data == value) {
            if (cur->prev) cur->prev->next = cur->next;
            else list->head = cur->next;
            if (cur->next) cur->next->prev = cur->prev;
            else list->tail = cur->prev;
            free(cur);
            return true;
        }
    }
    return false;
}
```

Complexity and common mistakes

- Finding the target: $O(n)$, same as singly
- Relinking, once found: $O(1)$ either direction
- Mistake: forgetting to update `tail` after the tail moves
- Mistake: skipping the new node's `prev` link

Mini-quiz

Why does a doubly list's `delete_value` need **no** separate `prev` -tracking variable, unlike the singly version?

Answer

Every node already stores its own `prev`.

The singly version had to track `prev` by hand while scanning; the doubly version just reads `cur->prev` directly.

8. Circular Lists and the Josephus Problem

A question to start

What if the **last** node's **next** pointed back at the first node, instead of **NULL** ?

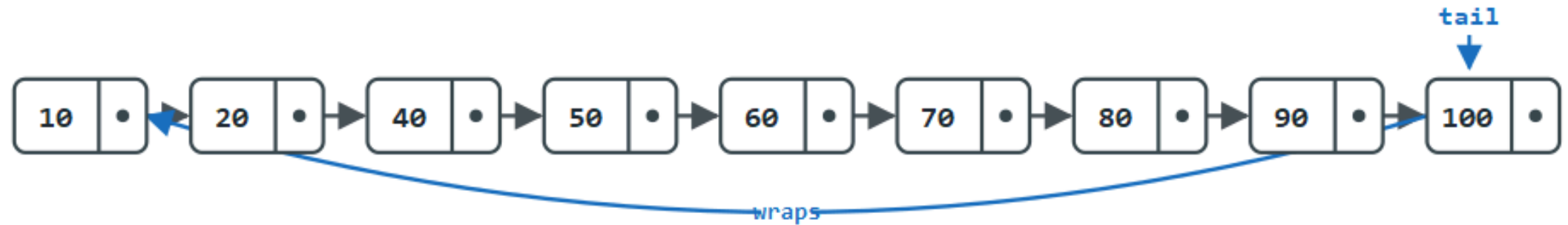
Intuition — people seated in a circle

- No "first" seat and no "last" seat, really
- Walk far enough and you are back where you started
- A circular list has no `NULL` to stop a walk
- Something else — a count, a condition — must stop it

One pointer, no separate head

- `tail` points at the last-inserted node
- `tail->next` is the head — no extra field
- A single node points at **itself** — a tiny loop
- Deleting needs a forward scan — there is no `prev`

Circular-list operations, step by step



Done: 2 laps, 18 steps in total (9 nodes x 2), 1 deletion (0 not found). Traversal: 10, 20, 40, 50, 60, 70, 80, 90, 100, 10, 20, 40, 50, 60, 70, 80, 90, 100.

Edge case — the last node, deleted

(empty)

```
traverse(NULL, 5): there is nothing to show.
```

Code — insert_tail()

```
Node *insert_tail(Node *tail, int value) {  
    Node *n = malloc(sizeof(Node));  
    n->data = value;  
    if (tail == NULL) {  
        n->next = n;      /* points at itself */  
        return n;  
    }  
    n->next = tail->next; /* new node -> old head */  
    tail->next = n;      /* old tail -> new node */  
    return n;  
}
```

Complexity and common mistakes

- Insert at the tail: $O(1)$, same trick as before
- Delete by value: $O(n)$ scan — no `prev` link
- Mistake: checking `cur == NULL` to stop a traversal
- A circular list needs a **step count**, not a `NULL`

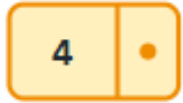
A short history

- **c. 67 CE** — Flavius Josephus, siege of Yodfat
- Legend: 41 soldiers, every 3rd one eliminated
- Josephus reportedly placed himself at the survivor's spot
- A single circular list solves the whole puzzle today

Every k -th person is eliminated

- n people stand in a circle, numbered 1.. n
- Count $k-1$ steps forward from the last survivor
- Eliminate the person you land on
- Repeat until only **one** person remains

The Josephus problem, step by step



Done: person 4 survives. Elimination order: 3, 6, 9, 2, 7, 1, 8, 5, 10. (Aside: the closed form -- $J(1)=0$, $J(n)=(J(n-1)+k) \bmod n$ -- gives the same survivor without simulating every step.)

Edge case — a circle of one



$n = 1$: the circle has just one person, remaining > 1 is never true -- the loop never runs, $\text{cur} \rightarrow \text{id}(1)$ is returned right away.

Code — josephus()

```
int josephus(int n, int k) {
    Node *cur = head;
    int remaining = n;
    while (remaining > 1) {
        for (int s = 1; s < k; s++) { /* k-1 steps */
            prev = cur;
            cur = cur->next;
        }
        prev->next = cur->next; /* remove cur */
        cur = prev->next;
        remaining--;
    }
    return cur->id; /* the survivor */
}
```

Complexity

- Simulating directly: $O(n \cdot k)$ in the worst case
- A closed-form recurrence gives the survivor in $O(n)$
- $J(1)=0$, $J(n) = (J(n-1) + k) \bmod n$

Mini-quiz

With $k = 1$, every count is just "the next person". Who survives a circle of 10?

Answer

Person 10 — the last one in line. With $k=1$ there is no skipping at all: elimination just proceeds in plain order, 1 through 9.

9. XOR Linked Lists: A Curiosity

A question to start

A doubly list spends two pointer fields per node. Could **one** field somehow hold both neighbors at once?

$\text{npx} = \text{prev} \text{ XOR } \text{next}$

- Store **one** field: `prev` 's address XOR `next` 's address
- Arriving from a known neighbor `known`, recover the other:
- `other = npx XOR known`
- XOR quietly "cancels" the neighbor you came from

XOR list traversal, step by step

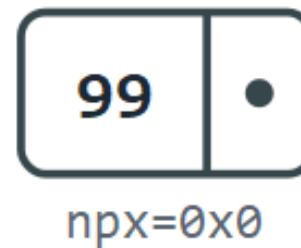
Every node has just ONE field: `npx = prev XOR next`.



Done. Forward: 10, 20, 30, 40, 50, 60, 70, 80, 90, 100. Backward: 100, 90, 80, 70, 60, 50, 40, 30, 20, 10.
Two-way traversal from a single npx field -- the cost is one XOR per step.

Edge case — inserting at the tail of an empty list

Every node has just ONE field: ``npx = prev XOR next``.



Done. Forward: 99. Backward: 99. Two-way traversal from a single npx field -- the cost is one XOR per step.

Code — the struct and the XOR trick

```
typedef struct Node {  
    int data;  
    uintptr_t npx;    /* XOR of prev and next */  
} Node;  
  
static Node *xor_node(uintptr_t npx, Node *known) {  
    return (Node *) (npx ^ (uintptr_t) known);  
}
```

Code — `traverse_forward()`

```
void traverse_forward(Node *head) {  
    Node *prev = NULL, *cur = head;  
    while (cur != NULL) {  
        printf(" %d", cur->data);  
        Node *next = xor_node(cur->npx, prev);  
        prev = cur; cur = next;  
    }  
}
```

Complexity

- Insert at head or tail: **$O(1)$** , same as before
- Every traversal step: one extra **XOR** — still $O(1)$ each
- Memory saved: one pointer field per node

Why this is a curiosity, not a habit

- C does **not** guarantee pointer↔integer round-trips this way
- `uintptr_t` casts back to a pointer are **implementation-defined**
- A garbage-collected language (Java) cannot do this at all
- Real code: use a plain doubly linked list instead

Mini-quiz

Why can a garbage collector never support an XOR linked list the way C's `malloc` can?

Answer

A GC needs to find every live pointer to trace and possibly move it. An address hidden inside an XOR'd integer is invisible to that scan — the GC cannot see it as a pointer.

10. Skip Lists

A question to start

A sorted array gets binary search, $O(\log n)$.
A sorted linked list cannot jump to the middle. Is $O(\log n)$ search possible anyway?

A short history

- **1990** — William Pugh, University of Maryland
- A randomized alternative to balanced search trees
- Each key gets a random "coin-flip" height
- Simpler to implement correctly than a balanced tree

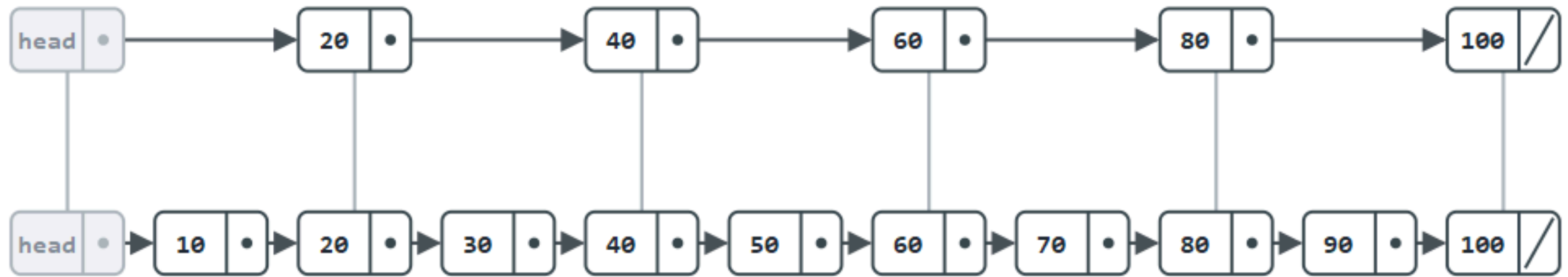
Intuition — a local road and an express lane

- Level 0: the full sorted list — every key
- Level 1: an **express lane** — only some keys
- Start on the express lane; drop down when it overshoots
- Fewer stops than checking every key one by one

Search: go right, or drop down

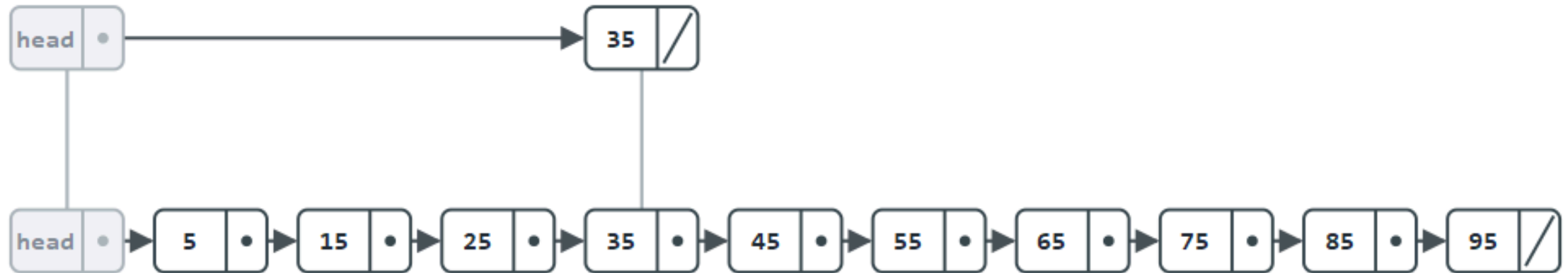
- At the current level, is `cur->forward[i]->value < target` ?
- Yes: step right, staying on this level
- No: **drop down** one level and try again
- Reaching level 0 and stepping once more: the answer

Skip list search, step by step



Done: 10 keys, 2 searches. Base list (sorted): 10, 20, 30, 40, 50, 60, 70, 80, 90, 100.

Edge case — only one node on the express lane



Done: 10 keys, 2 searches. Base list (sorted): 5, 15, 25, 35, 45, 55, 65, 75, 85, 95.

Code — `sl_search()`

```
int sl_search(SkipList *sl, int value, int *cmp) {
    Node *cur = sl->header;
    for (int i = MAX_LEVEL-1; i >= 0; i--) {
        while (cur->forward[i] &&
               cur->forward[i]->value < value) {
            cur = cur->forward[i];    /* go right */
            (*cmp)++;
        }
        /* ... else: drop down one level ... */
    }
    cur = cur->forward[0];
    return cur && cur->value == value;
}
```

Complexity and common mistakes

- Enough express levels: $O(\log n)$ expected search
- Every key stuck at level 0 only: degrades to $O(n)$
- Mistake: stopping instead of dropping down a level
- This demo's levels are fixed, not really flipped live

Mini-quiz

If every key in a skip list happened to get level 1 only (no express lane at all), what does search cost, in Big-O?

Answer

$O(n)$. With no express lane, level 0 is the only level left — search degrades to the same linear scan as a plain linked list.

11. Arrays vs. Linked Lists

Arrays vs. linked lists

Operation	Array	Linked list
Access by index	$O(1)$	$O(n)$
Insert/delete at front	$O(n)$	$O(1)$
Search by value	$O(n)$	$O(n)$

When to choose which

- Need `arr[k]` constantly? Arrays win outright
- Frequent front-inserts, unknown final size? Lists win
- Cache-friendly, contiguous scans? Arrays win
- Never shift existing data on insert? Lists win

Mini-quiz

You are building a stack that only ever grows and shrinks at one end. Array or list?

Answer

Either works well — both give $O(1)$ at one end: a dynamic array's `append`, or a list's `insert_head`. The real deciding factor is whether you also need `arr[k]`.

Summary — arrays and matrices

Idea	Key fact
Array insert/delete	Shifting; $O(n)$ worst, $O(1)$ at the end
Dynamic array	Doubling; $O(1)$ amortized append
Row-major layout	<code>i*COLS+j</code> ; match the walk to the layout
Sparse matrix	Triplets; fast transpose $O(\text{nnz} + \text{COLS})$

Summary — linked lists

Idea	Key fact
Singly list	insert_head $O(1)$; insert_after: order matters
Doubly / circular	Two links; <code>tail->next</code> is the head
Josephus	Circular list, one bypass per elimination
XOR / skip list	One-field trick; randomized $O(\log n)$

The big picture

Arrays trade flexible insertion for $O(1)$ indexing; linked lists trade indexing for $O(1)$ insertion anywhere you already are. Every structure this week picks one side of that same trade-off.

Self-check round

Four short questions. Think before the answer appears on the next slide. Full exercises are in the week notes.

1. Inserting at index 0 in a 20-element array: how many elements move?

All 20 — the entire array shifts right by one slot.

2. Why is a dynamic array's append still called $O(1)$, if growing costs $O(n)$?

The $O(n)$ growth is rare, and its cost **amortizes**: averaged over n appends, the total stays under $2n$.

3. In `insert_after`, why must `n->next = prev->next` happen before `prev->next = n`?

Reversed, `prev->next` already equals `n` by the time step one reads it — the new node ends up pointing at itself.

4. Why does a skip list's search cost degrade to $O(n)$ with no express lane?

With no upper level, level 0 is the only level left — search becomes the same one-key-at-a-time walk as a plain linked list.

Next week

Week 3 — Stacks and Queues

LIFO and FIFO built directly on the array and linked-list tools from this week — the same nodes, the same shifting array, now disciplined into one entry/exit point.

References (1/2)

- Course syllabus, Week 2: `docs/syllabus/syllabus.en.md`
- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed. MIT Press
- Sedgewick, Wayne. *Algorithms*, 4th ed. Addison-Wesley
- Knuth. *The Art of Computer Programming, Vol. 1*, 3rd ed.

References (2/2)

- Pugh (1990) — skip lists
- McCarthy (1956) — Lisp cons cells
- Flavius Josephus, *The Jewish War* — the elimination problem
- [williamfiset/Algorithms](#) · Programiz DSA